

Лабораторная работа № 13: Исследование методов создания колбэков и промисов

Цель работы: Сформировать умение анализировать методы создания колбэков и промисов.

1. Подготовка рабочего окружения

1.1 Обновление основной ветки

1.2 Создание новой ветки для работы

Задание 1: Создание и сравнение колбэков (Базовое)

Задача: Создать модуль для работы с файлами, использующий колбэки.

Создание файла `fileOperations.js`

Создайте файл `fileOperations.js` со следующим содержимым:

javascript

```
const fs = require('fs');
const path = require('path');

/**
 * Класс для работы с файлами с использованием колбэков
 */
class FileManager {
  /**
   * Конструктор
   * @param {string} baseDir - базовая директория для операций
   */
  constructor(baseDir = './data') {
    this.baseDir = baseDir;

    // Создаём директорию, если её нет (синхронно для простоты)
    if (!fs.existsSync(baseDir)) {
      fs.mkdirSync(baseDir, { recursive: true });
    }
  }
}
```

```

        console.log(`Создана директория: ${baseDir}`);
    }
}

/**
 * Создание файла с содержимым (колбэк)
 * @param {string} filename - имя файла
 * @param {string} content - содержимое
 * @param {Function} callback - (err, filePath) => void
 */
createFile(filename, content, callback) {
    const filePath = path.join(this.baseDir, filename);

    fs.writeFile(filePath, content, 'utf8', (err) => {
        if (err) {
            callback(err, null);
            return;
        }
        callback(null, filePath);
    });
}

/**
 * Чтение файла (колбэк)
 * @param {string} filename - имя файла
 * @param {Function} callback - (err, content) => void
 */
readFile(filename, callback) {
    const filePath = path.join(this.baseDir, filename);

    fs.readFile(filePath, 'utf8', (err, data) => {
        if (err) {
            callback(err, null);
            return;
        }
        callback(null, data);
    });
}

/**
 * Получение информации о файле (колбэк)
 * @param {string} filename - имя файла
 * @param {Function} callback - (err, stats) => void
 */

```

```

getFileStats(filename, callback) {
  const filePath = path.join(this.baseDir, filename);

  fs.stat(filePath, (err, stats) => {
    if (err) {
      callback(err, null);
      return;
    }
    callback(null, {
      size: stats.size,
      created: stats.birthtime,
      modified: stats.mtime,
      isFile: stats.isFile()
    });
  });
}

```

```

/**
 * Удаление файла (колбэк)
 * @param {string} filename - имя файла
 * @param {Function} callback - (err) => void
 */
deleteFile(filename, callback) {
  const filePath = path.join(this.baseDir, filename);

  fs.unlink(filePath, (err) => {
    if (err) {
      callback(err);
      return;
    }
    callback(null);
  });
}

```

```

/**
 * Список файлов в директории (колбэк)
 * @param {Function} callback - (err, files) => void
 */
listFiles(callback) {
  fs.readdir(this.baseDir, (err, files) => {
    if (err) {
      callback(err, null);
      return;
    }
  })
}

```

```

// Фильтруем только файлы (не директории)
const filePromises = files.map(file => {
  return new Promise((resolve) => {
    const filePath = path.join(this.baseDir, file);
    fs.stat(filePath, (err, stats) => {
      resolve({ name: file, isFile: !err && stats.isFile() });
    });
  });
});

Promise.all(filePromises)
  .then(results => {
    const onlyFiles = results
      .filter(r => r.isFile)
      .map(r => r.name);
    callback(null, onlyFiles);
  })
  .catch(err => callback(err, null));
});
}
}

```

```
module.exports = FileManager;
```

Создание тестового скрипта

Создайте файл `test-callbacks.js`:

```
javascript
```

```
const FileManager = require('./fileOperations');
```

```
// Создаём экземпляр менеджера
```

```
const fileManager = new FileManager('./test-data');
```

```
console.log('=== ТЕСТИРОВАНИЕ КОЛБЭКОВ ===\n');
```

```
// 1. Создание файла
```

```
console.log('1. Создание файла...');
```

```
fileManager.createFile('test1.txt', 'Привет, мир!', (err, filePath) => {
  if (err) {
```

```

    console.error(' ❌ Ошибка создания:', err.message);
    return;
}
console.log(' ✅ Файл создан: ${filePath}');

// 2. Чтение файла (вложенный колбэк)
console.log('\n2. Чтение файла...');
fileManager.readFile('test1.txt', (err, content) => {
    if (err) {
        console.error(' ❌ Ошибка чтения:', err.message);
        return;
    }
    console.log(' ✅ Содержимое: "${content}"');

// 3. Получение статистики (ещё глубже)
    console.log('\n3. Получение статистики...');
    fileManager.getFileStats('test1.txt', (err, stats) => {
        if (err) {
            console.error(' ❌ Ошибка статистики:', err.message);
            return;
        }
        console.log(' ✅ Статистика:');
        console.log('   Размер: ${stats.size} байт');
        console.log('   Создан: ${stats.created}');
        console.log('   Изменён: ${stats.modified}');

// 4. Создание второго файла
        console.log('\n4. Создание второго файла...');
        fileManager.createFile('test2.txt', 'Второй файл для демонстрации', (err, filePath2)
=> {
            if (err) {
                console.error(' ❌ Ошибка создания второго файла:', err.message);
                return;
            }
            console.log(' ✅ Второй файл создан: ${filePath2}');

// 5. Список файлов
            console.log('\n5. Список файлов...');
            fileManager.listFiles((err, files) => {
                if (err) {
                    console.error(' ❌ Ошибка получения списка:', err.message);
                    return;
                }
                console.log(' ✅ Файлы в директории:');

```


Создание файла fileOperationsPromises.js

Создайте файл fileOperationsPromises.js:

javascript

```
const fs = require('fs');
const path = require('path');
const util = require('util');

// Преобразуем методы fs в промисы
const readFile = util.promisify(fs.readFile);
const writeFile = util.promisify(fs.writeFile);
const unlink = util.promisify(fs.unlink);
const readdir = util.promisify(fs.readdir);
const stat = util.promisify(fs.stat);
const mkdir = util.promisify(fs.mkdir);

class FileManagerPromises {
  constructor(baseDir = './data-promises') {
    this.baseDir = baseDir;
    this.initDir();
  }

  /**
   * Инициализация директории (синхронно для простоты)
   */
  initDir() {
    if (!fs.existsSync(this.baseDir)) {
      fs.mkdirSync(this.baseDir, { recursive: true });
      console.log(`Создана директория: ${this.baseDir}`);
    }
  }

  /**
   * Создание файла (промис)
   * @param {string} filename - имя файла
   * @param {string} content - содержимое
   * @returns {Promise<string>} - путь к созданному файлу
   */
  async createFile(filename, content) {
    const filePath = path.join(this.baseDir, filename);
    await writeFile(filePath, content, 'utf8');
    return filePath;
  }
}
```

```
}
```

```
/**
```

```
 * Чтение файла (промис)
```

```
 * @param {string} filename - имя файла
```

```
 * @returns {Promise<string>} - содержимое файла
```

```
 */
```

```
async readFile(filename) {  
    const filePath = path.join(this.baseDir, filename);  
    return await readFile(filePath, 'utf8');  
}
```

```
/**
```

```
 * Получение информации о файле (промис)
```

```
 * @param {string} filename - имя файла
```

```
 * @returns {Promise<Object>} - статистика файла
```

```
 */
```

```
async getFileStats(filename) {  
    const filePath = path.join(this.baseDir, filename);  
    const stats = await stat(filePath);  
    return {  
        size: stats.size,  
        created: stats.birthtime,  
        modified: stats.mtime,  
        isFile: stats.isFile()  
    };  
}
```

```
/**
```

```
 * Удаление файла (промис)
```

```
 * @param {string} filename - имя файла
```

```
 * @returns {Promise<void>}
```

```
 */
```

```
async deleteFile(filename) {  
    const filePath = path.join(this.baseDir, filename);  
    await unlink(filePath);  
}
```

```
/**
```

```
 * Список файлов (промис)
```

```
 * @returns {Promise<string[]>} - массив имён файлов
```

```
 */
```

```
async listFiles() {  
    const files = await readdir(this.baseDir);
```

```

const fileStats = await Promise.all(
  files.map(async (file) => {
    const filePath = path.join(this.baseDir, file);
    const stats = await stat(filePath);
    return { name: file, isFile: stats.isFile() };
  })
);
return fileStats.filter(f => f.isFile).map(f => f.name);
}

/**
 * Создание нескольких файлов параллельно
 * @param {Array<{filename: string, content: string}>} files
 * @returns {Promise<string[]>} - массив путей
 */
async createMultipleFiles(files) {
  const promises = files.map(({ filename, content }) =>
    this.createFile(filename, content)
  );
  return await Promise.all(promises);
}

/**
 * Чтение нескольких файлов параллельно
 * @param {string[]} filenames
 * @returns {Promise<Object>} - объект { filename: content }
 */
async readMultipleFiles(filenames) {
  const promises = filenames.map(async (filename) => {
    const content = await this.readFile(filename);
    return { [filename]: content };
  });
  const results = await Promise.all(promises);
  return Object.assign({}, ...results);
}
}

```

```
module.exports = FileManagerPromises;
```

Создание тестового скрипта

Создайте файл `test-promises.js`:

javascript

```
const FileManagerPromises = require('./fileOperationsPromises');

const fileManager = new FileManagerPromises('./test-data-promises');

async function testFileOperations() {
  console.log('=== ТЕСТИРОВАНИЕ ПРОМИСОВ ===\n');

  try {
    // 1. Создание файла
    console.log('1. Создание файла...');
    const filePath = await fileManager.createFile('test1.txt', 'Привет из промисов!');
    console.log(` ✅ Файл создан: ${filePath}`);

    // 2. Чтение файла
    console.log('\n2. Чтение файла...');
    const content = await fileManager.readFile('test1.txt');
    console.log(` ✅ Содержимое: "${content}"`);

    // 3. Получение статистики
    console.log('\n3. Получение статистики...');
    const stats = await fileManager.getFileStats('test1.txt');
    console.log(` ✅ Статистика:`);
    console.log(`   Размер: ${stats.size} байт`);
    console.log(`   Создан: ${stats.created}`);
    console.log(`   Изменён: ${stats.modified}`);

    // 4. Создание нескольких файлов параллельно
    console.log('\n4. Создание нескольких файлов параллельно...');
    const files = [
      { filename: 'test2.txt', content: 'Второй файл' },
      { filename: 'test3.txt', content: 'Третий файл' },
      { filename: 'test4.txt', content: 'Четвёртый файл' }
    ];
    const paths = await fileManager.createMultipleFiles(files);
    console.log(` ✅ Создано файлов: ${paths.length}`);
    paths.forEach(p => console.log(`   - ${p}`));

    // 5. Список всех файлов
    console.log('\n5. Список файлов...');
    const fileList = await fileManager.listFiles();
    console.log(` ✅ Найдено файлов: ${fileList.length}`);
    fileList.forEach(f => console.log(`   - ${f}`));
  }
}
```

```

// 6. Чтение нескольких файлов параллельно
console.log("\n6. Чтение нескольких файлов параллельно...");
const contents = await fileManager.readMultipleFiles(fileList);
console.log(' ✅ Содержимое файлов:');
Object.entries(contents).forEach(([filename, content]) => {
  console.log(`   - ${filename}: "${content}"`);
});

// 7. Очистка (удаление всех файлов)
console.log("\n7. Очистка...");
for (const file of fileList) {
  await fileManager.deleteFile(file);
  console.log(`   ✅ ${file} удалён`);
}

console.log("\n✅ Все операции завершены!");
console.log('🌟 Код стал намного чище и читаемее!');

} catch (error) {
  console.error("\n❌ Ошибка:", error.message);
  console.error('Stack:', error.stack);
}
}

// Запуск
testFileOperations();

```

Сравнение подходов

Запустите оба скрипта и сравните:

```
bash
```

```
node test-callbacks.js
```

```
node test-promises.js
```

Задание 3: Смешанный подход (Сложное)

Задача: Создать гибридный модуль, поддерживающий оба стиля, и добавить обработку ошибок.

Задание 4: Исследование производительности (Бонусное)

Задача: Сравнить производительность синхронных, асинхронных (колбэки) и промис-операций.

Фиксация изменений

Добавить файлы в Git

Создать коммит

Отправить в удалённый репозиторий

Создать Pull Request